



Predicción de Condición de Egreso Hospitalario mediante Spark ML

Solemne I: Pipeline de Big Data y Modelado Predictivo bajo Metodología CRISP-DM

Integrantes: Cesar Lopez, Luciano Abarca, Daniel Paredes, Manuel Vidal, Vicente Poblete

Profesor: Danilo Gómez Correa | **Curso:** Ciencia de Datos | **Santiago, Chile, 2026**

- ❏ Esta presentación muestra el desarrollo completo de un pipeline de Big Data usando Apache Spark ML, siguiendo la metodología CRISP-DM, para predecir la condición de egreso hospitalario en Chile.



Fase 1: Entendimiento del Negocio

Problema y pregunta

Los egresos hospitalarios pueden terminar en alta en vida o fallecimiento. La pregunta clave es: **¿Qué características permiten predecir el resultado?**

Utilidad clínica:

- Priorización de recursos y triaje
- Alerta temprana para pacientes de mayor riesgo
- Apoyo a gestión hospitalaria y políticas públicas

Transformación a ML

Tipo de problema: Clasificación binaria

Variable objetivo:

- 0 = egreso vivo
- 1 = egreso fallecido

Métricas: AUC-ROC (principal), F1-Score, recall, precisión

- ❏ El problema se transforma en clasificación binaria, identificando pacientes con mayor probabilidad de egresar fallecidos, priorizando métricas robustas ante el desbalance de clases.

Fase 2: Entendimiento de los Datos

Dataset: EGRESOS_2024 | **Fuente:** Registros administrativos de egresos hospitalarios de Chile

1.048.575

Registros totales

Volumen que justifica procesamiento distribuido

15

Variables

Categorías, numéricas y códigos CIE-10

97.54%

Pacientes vivos

Fuerte desbalance de clases

2.46%

Pacientes fallecidos

Clase minoritaria crítica

Complejidad del dataset

- Variables categóricas: sexo, grupo etario, región, previsión, país de origen
- Variables numéricas: días de estada
- Códigos clínicos CIE-10 (DIAG1, DIAG2)
- Duplicados: 20.6% de registros
- Ausentes: DIAG2 con 90.02% de valores faltantes

Entorno técnico

Python 3.13 + Java 17 + PySpark 3.5

Configuración: Spark local[12] con 12 cores y 4 GB RAM

GPU: RTX 4060 para XGBoost

❏ El volumen del dataset supera el millón de registros, por lo que Spark permite manejar la carga, exploración, transformación y modelado de forma más eficiente que herramientas tradicionales como pandas.

EDA masiva: distribución y calidad de datos

Principales hallazgos de la exploración inicial

DIAS_ESTADA extremo

Media: 6.22 días | Desviación: 33.31 | **Máximo: 15.162 días**
(outlier extremo)

DIAG2 ausente

90.02% de valores ausentes en segunda diagnosis,
imposibilitando su uso predictivo

Duplicados completos

215.967 registros (20.6%) duplicados, requiriendo
eliminación

Desbalance crítico

Vivo: 1.022.746 registros (97.54%) | Fallecido: 25.829
registros (2.46%)

- ❏ El principal desafío detectado fue el fuerte desbalance de clases, ya que los fallecidos representan solo un 2.46% del total. Esto condicionó la elección de métricas y la estrategia de modelado.

Fase 3: Limpieza, features y pipeline de preparación

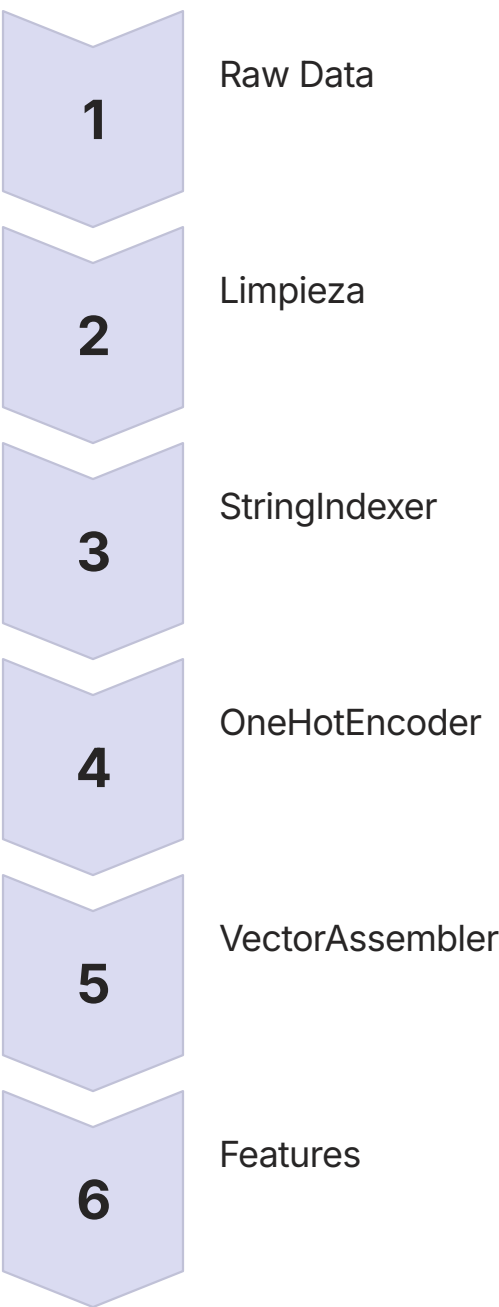
Pasos del pipeline

- 1. Filtrado: CONDICION_EGRESO = 1 o 2, ANO_EGRESO = 2024
- 2. Eliminación de duplicados completos
- 3. Creación de label binario
- 4. Ingeniería de características (CAPITULO_CIE10, TIENE_DIAG2, ES_CHILE)
- 5. Exclusión de DIAS_ESTADA como predictor

Variables predictoras

Categorías (8): Establecimiento, sexo, grupo edad, país, región, previsión, capítulo CIE-10

Numéricas (2): TIENE_DIAG2, ES_CHILE



```
df_clean = df

# Limpiar espacios en variables de texto
for c, tipo in df_clean.dtypes:
    if tipo == "string":
        df_clean = df_clean.withColumn(c, trim(col(c)))
        df_clean = df_clean.withColumn(c, when(col(c) == "", None).otherwise(col(c)))

# Mantener solo registros válidos para el problema
df_clean = df_clean.filter(col("CONDICION_EGRESO").isin([1.0, 2.0]))
df_clean = df_clean.filter(col("ANO_EGRESO") == 2024)
df_clean = df_clean.filter(col("DIAS_ESTADA").isNotNull())
df_clean = df_clean.filter(col("DIAS_ESTADA") >= 1)

print("Registros después de filtros básicos:", df_clean.count())
```

Registros después de filtros básicos: 1048575

Vector resultante

102 componentes finales tras OneHotEncoder y VectorAssembler

❏ La preparación fue encapsulada en un Pipeline de Spark ML, lo que permite reproducibilidad y evita fuga de información, ya que el ajuste se realizó solo sobre el conjunto de entrenamiento.

Separación de datos y ponderación por clase

Separación train/test

Train: 80% (838.608 registros)

Test: 20% (209.967 registros)

Seed: 2026 (reproducible)

Distribución de clases

Conjunto	Total	Vivos	Fallecidos
Train	838.608	817.916	20.692
Test	209.967	204.830	5.137

Pesos de clase

0.51

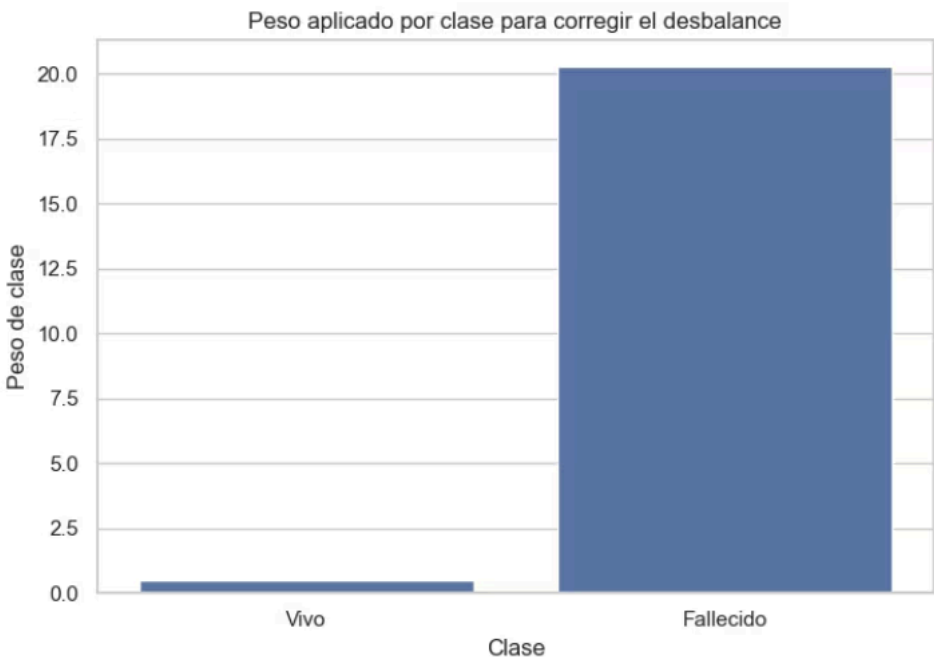
Clase viva

Peso bajo

20.26

Clase fallecida

Peso alto



El desbalance hace que accuracy no sea suficiente. Se usa **weightCol** para penalizar más los errores sobre la clase minoritaria, más escalable que sobremuestreo.

❏ Al usar pesos de clase, el modelo aprende a darle más importancia a los casos de fallecimiento, sin duplicar registros ni romper la lógica distribuida de Spark.

Fase 4: Modelos entrenados

Se implementaron dos enfoques principales dentro de Spark ML

Enfoque A: Regresión Logística

Variantes: L2/Ridge, L1/Lasso, Elastic Net

- Interpretable: permite analizar coeficientes
- Rápida: entrenamiento eficiente
- Lineal: relaciones lineales entre features

Enfoque B: Random Forest

Parámetros: numTrees, maxDepth

- Robusto: captura relaciones no lineales
- No lineal: mejor equilibrio operativo
- Feature importance: variables relevantes

Modelo complementario: XGBoost con GPU

AUC-ROC: 0.8638 | **Recall fallecido:** 0.83

No se considera modelo principal porque requirió convertir datos Spark a NumPy, perdiendo escalabilidad distribuida.

Elección técnica

Contraste entre **interpretabilidad** (logística) y **capacidad predictiva** (Random Forest)

- ❏ Se compararon modelos lineales regularizados contra modelos de árboles. Esto permite contrastar interpretabilidad contra capacidad de capturar relaciones no lineales.

Resultados iniciales y variables relevantes

Comparación de modelos base

Modelo	AUC-ROC	F1-Score	Recall fallecidos	Accuracy
Regresión L2	0.8570	0.8430	0.7946	0.7602
Regresión L1	0.7249	0.8830	0.5474	0.8236
Elastic Net	0.8120	0.8627	0.6282	0.7907
Random Forest	0.8480	0.8463	0.7689	—

Variables de mayor riesgo

1. GRUPO_EDAD = 85 A MÁS
2. GRUPO_EDAD = 80 A 84 AÑOS
3. CAPITULO_CIE10 = J (respiratorias)
4. CAPITULO_CIE10 = I (circulatorias)
5. CAPITULO_CIE10 = A (infecciosas)

Variables de menor riesgo

- CAPITULO_CIE10 = O (embarazo, parto, puerperio)
- Grupos etarios jóvenes (0-19 años)

Los predictores son **clínicamente coherentes**

❏ La regresión L2 logró el mayor recall base para fallecidos, mientras que Random Forest obtuvo un mejor balance general. Además, los predictores más importantes son clínicamente coherentes.

Fase 5: Validación cruzada y comparación final

01

CrossValidator con 5 folds

Métrica de selección: AUC-ROC

02

Grid Regresión Logística L1

regParam: 0.001, 0.01, 0.1 | maxIter: 50, 100 | **Mejor: regParam=0.001, maxIter=100**

03

Grid Random Forest

numTrees: 50, 100 | maxDepth: 5, 10 | **Mejor: numTrees=100, maxDepth=10**

Regresión Logística L1 CV

AUC-ROC: 0.8602

F1-Score: 0.8337

Recall ponderado: 0.7461

Precisión ponderada: 0.9713

Tiempo: 217.5 segundos

Random Forest CV

AUC-ROC: 0.8511

F1-Score: 0.8450

Recall ponderado: 0.7631

Precisión ponderada: 0.9705

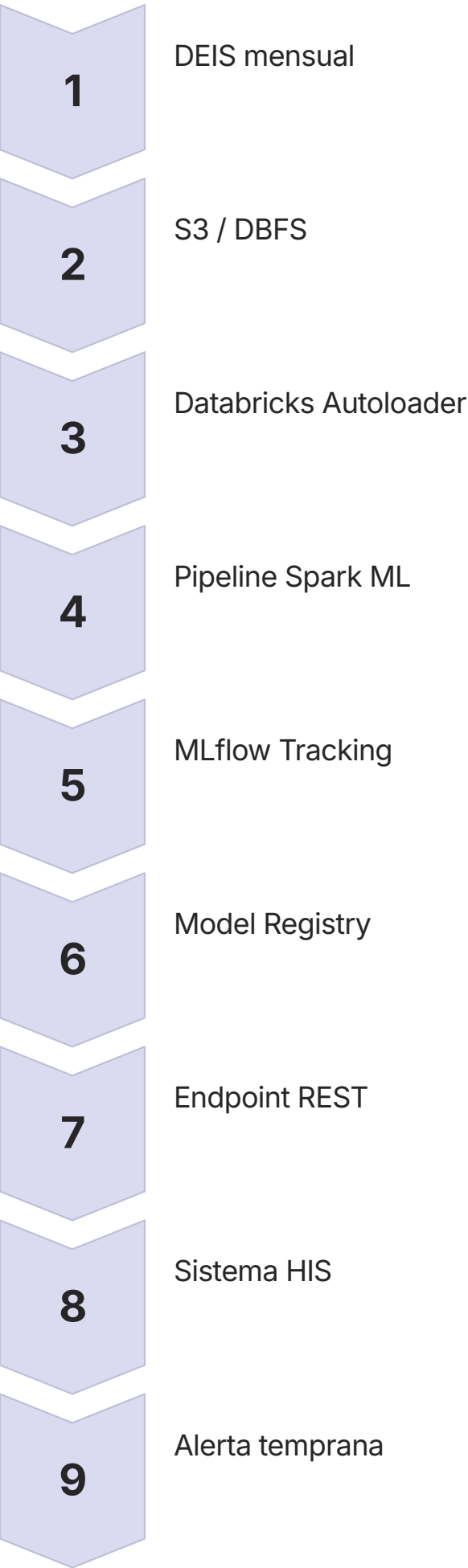
Tiempo: 389.6 segundos

 **Conclusión técnica:** Aunque LR L1 CV obtuvo mayor AUC-ROC (0.8602), Random Forest CV obtuvo mejor F1-Score (0.8450) y mejor balance operativo. **Se selecciona Random Forest como modelo principal** para el problema clínico.

 La validación cruzada permitió ajustar los hiperparámetros de forma rigurosa. Aunque la regresión logística obtuvo mayor AUC, Random Forest presentó mejor equilibrio operativo para el problema clínico.

Fase 6: Despliegue, conclusión y valor de negocio

Arquitectura de producción propuesta



Escalabilidad

- Pipeline encapsulado en Spark ML
- Datos en Parquet
- Modelos como PipelineModel
- Transferible a clúster real

Infraestructura recomendada

1. **Databricks:** MLflow, Delta Lake, Autoloader, Model Registry, Model Serving
2. **AWS EMR + SageMaker:** Alternativa flexible

Conclusión técnica

Random Forest detecta ~77 de cada 100 fallecidos (recall 76.31%).
Solución escalable para priorización clínica y seguimiento intensivo.

Valor de negocio

Apoyo a gestión hospitalaria, camas UCI, alerta temprana. **No reemplaza decisión médica**, actúa como sistema de apoyo.

Próximos pasos

Reentrenamiento mensual, ajuste umbral, monitoreo drift, validación clínica con expertos médicos.

"Spark ML permite transformar registros administrativos masivos en una herramienta escalable de apoyo a la gestión hospitalaria."

❑ El proyecto demuestra que un pipeline distribuido puede ser aplicado a datos reales de salud para generar valor clínico y operativo, especialmente cuando se usa como apoyo a la toma de decisiones.